UNIT V : Analytical Questions

Slot C

Name : K. Lahari **Reg No. : 192325118**
Course : Cryptography And Network Security **Course Code : CSA5104**

Q1. Web communication threats and the security services that counter them

Step 1 — Objective: Identify the major threats that arise in web communication and map each one to the security service (confidentiality, integrity, authentication, non-repudiation) and mechanism that mitigates it.

Step 2 — Confidentiality threats: Eavesdropping/sniffing on an unencrypted channel exposes credentials and data in transit. Mitigation: encryption of the session, typically via SSL/TLS using a negotiated symmetric cipher.

Step 3 — Integrity threats: Packet tampering or injection can silently alter data in transit. Mitigation: keyed hashes/MACs (HMAC) or, in modern TLS, authenticated encryption (AEAD) that detects any modification.

Step 4 — Authentication threats: Masquerading, phishing, and session hijacking let an attacker impersonate a legitimate party. Mitigation: CA-issued digital certificates for server (and optionally client) authentication, combined with strong user authentication such as passwords plus MFA.

Step 5 — Non-repudiation threats: A party may later deny having sent or authorised a transaction. Mitigation: digital signatures bound to a certified private key, optionally combined with trusted timestamping, so authorship cannot be credibly denied.

Step 6 — Threat-to-mechanism map:

Security goal	Representative attack	Mitigating mechanism
Confidentiality	Eavesdropping / sniffing	TLS encryption (AES-GCM, ChaCha20)
Integrity	Packet tampering / injection	HMAC / AEAD tag verification
Authentication	Masquerading / phishing / MITM	Digital certificates + CA chain, MFA
Non-repudiation	Denial of a sent transaction	Digital signatures + timestamping

Step 7 — Conclusion: No single control covers all four goals; TLS supplies confidentiality and integrity for the channel, PKI-based certificates supply authentication, and digital signatures supply non-repudiation. Web security is therefore inherently a layered combination of these mechanisms.

Q2. SSL vs TLS — architecture, mechanisms, handshake, and attack resistance

Step 1 — Objective: Compare SSL and TLS across architecture, cryptographic mechanisms, handshake behaviour, and resistance to known attacks, and justify TLS's preference.

Step 2 — Architecture: Both protocols share the same layered design — a Record Protocol carrying data, and upper sub-protocols (Handshake, Change Cipher Spec, Alert) that negotiate and manage the secure session. TLS keeps this structure but tightens the specification and key-derivation process.

Step 3 — Handshake differences: SSL derives keys with an ad hoc, protocol-specific construction. TLS replaces this with a standard HMAC-based pseudorandom function (PRF) for key derivation, and later adds extensions (SNI, ALPN) and, in TLS 1.3, a shortened one-round-trip handshake.

Step 4 — Cryptographic algorithms: SSL 3.0 is largely limited to weak/legacy ciphers such as RC4, DES/3DES, and MD5. TLS supports modern authenticated-encryption ciphers (AES-GCM, ChaCha20-Poly1305), SHA-256 and above, and mandates ephemeral Diffie–Hellman (ECDHE) for forward secrecy in TLS 1.3.

Step 5 — Resistance to attacks: SSL (and early TLS versions) are vulnerable to POODLE, BEAST, and cipher-downgrade attacks rooted in weak block-cipher modes and CBC padding. TLS 1.2 mitigates these with AEAD ciphers, and TLS 1.3 removes weak cipher suites and static RSA key exchange entirely, closing most of these avenues.

Step 6 — Comparison table:

Property	**SSL (2.0/3.0)**	**TLS (1.2/1.3)**
Key derivation	Ad hoc construction	HMAC-based PRF
Cipher support	RC4, DES/3DES, MD5	AES-GCM, ChaCha20, SHA-256+
Forward secrecy	Optional/rare	Mandatory in TLS 1.3 (ECDHE)
Known weaknesses	POODLE, BEAST, weak ciphers	Largely closed (1.3 removes legacy modes)
Handshake round trips	2 RTT	1 RTT (0-RTT resumption in 1.3)

Step 7 — Conclusion: TLS is preferred because it standardises key derivation, supports strong authenticated-encryption ciphers, mandates forward secrecy, and has systematically eliminated the weaknesses that made SSL exploitable — while also being faster to negotiate.

Q3. The SSL/TLS handshake — establishing a secure channel

Step 1 — Objective: Model the sequence of messages a client and server exchange to authenticate each other and derive a shared session key.

Step 2 — ClientHello: The client sends the TLS versions and cipher suites it supports, along with a random nonce, opening the negotiation.

Step 3 — ServerHello and certificate: The server replies with its chosen cipher suite, its own random nonce, and its digital certificate containing its public key and a chain back to a trusted Certificate Authority (CA).

Step 4 — Certificate verification and key exchange: The client validates the certificate chain, expiry, and hostname against its trust store. Key material (a premaster secret) is then established — either encrypted under the server's public key (RSA key exchange) or derived through an ephemeral Diffie–Hellman exchange (ECDHE) for forward secrecy.

Step 5 — Session key derivation: Both sides feed the premaster secret and the two random nonces into the PRF to derive identical symmetric session keys, without ever transmitting the keys themselves.

Step 6 — Finished messages: Each side sends a Finished message — a MAC over the entire handshake transcript, encrypted with the new session keys — confirming that both parties derived matching keys and that the handshake was not tampered with.

Step 7 — Role of certificates vs session keys:

Element	Role
Digital certificate	Binds the server's identity to its public key; verified against a trusted CA to authenticate the server
Session key	A short-lived symmetric key derived from the handshake, used to encrypt and integrity-protect the bulk application data efficiently

Step 8 — Conclusion: Certificates provide the identity/authentication anchor while session keys provide efficient, ongoing confidentiality and integrity — together they let two previously unacquainted parties establish a trusted, encrypted channel over an untrusted network.

Q4. Online banking over an unsecured network — how TLS protects the session

Step 1 — Objective: Analyse how SSL/TLS protects a banking user's credentials and transaction data when the underlying network (e.g., public Wi-Fi) is unsecured.

Step 2 — Threat model: An attacker on the same unsecured network can passively sniff traffic or actively position itself as a man-in-the-middle (MITM) between the user and the bank's server.

Step 3 — Confidentiality protection: TLS encrypts the entire application-layer payload — login credentials, account numbers, transaction amounts — so any packets captured by the attacker are ciphertext, not usable plaintext.

Step 4 — Integrity protection: The AEAD cipher's authentication tag (or a separate MAC) means any attempt to alter a transaction amount or destination account in transit is detected and the connection is dropped.

Step 5 — Authentication protection: The bank's certificate, verified against a trusted CA, prevents an attacker from transparently substituting its own server; a MITM without a certificate the browser trusts triggers a certificate warning rather than a silent redirection.

Step 6 — Forward secrecy: Ephemeral key exchange (ECDHE) means that even if a long-term key were later compromised, past captured sessions cannot be decrypted, limiting the blast radius of any future compromise.

Step 7 — Conclusion: The insecurity of the underlying network is irrelevant to a correctly established TLS session — encryption defeats eavesdropping, the MAC/AEAD tag defeats tampering, and certificate validation defeats impersonation, so the user's credentials and transaction data remain protected end-to-end.

Q5. Digital certificates in SSL/TLS and the risk of accepting an invalid one

Step 1 — Objective: Explain how digital certificates are used in SSL/TLS and analyse the consequences of a client accepting an invalid, expired, or self-signed certificate without verification.

Step 2 — Certificate structure and CA role: A certificate binds a public key to an identity (domain name/organisation), and is digitally signed by a trusted Certificate Authority whose own key the browser already trusts, forming a chain of trust.

Step 3 — Verification steps: A correctly behaving client checks the signature chain up to a trusted root, the validity period (not expired/not yet valid), revocation status (CRL/OCSP), and that the certificate's subject matches the requested hostname.

Step 4 — Risk of accepting an expired or otherwise invalid certificate: Expiry and chain checks exist because a certificate's guarantees are time- and issuer-bound; bypassing them means the client can no longer be sure the public key it is using still belongs to, or was ever verified for, the server it thinks it is talking to.

Step 5 — Risk of accepting a self-signed certificate blindly: A self-signed certificate has no third-party attestation of identity, so an attacker can generate one for any name and present it during a MITM attack; a client that accepts it without out-of-band verification cannot distinguish the real server from the attacker.

Step 6 — Consequences:

Scenario accepted without verification	Resulting risk
Expired certificate	No assurance the key/identity binding is still valid; may mask a compromised or rotated key
Self-signed certificate	No independent identity verification; enables undetected MITM impersonation
Hostname mismatch ignored	Client may be talking to the wrong server entirely

Step 7 — Conclusion: Certificate validation is the trust anchor of the entire TLS authentication guarantee; skipping any check collapses that guarantee and re-exposes the session to exactly the impersonation and eavesdropping attacks TLS is meant to prevent.

Q6. Symmetric vs asymmetric cryptography in SSL/TLS

Step 1 — Objective: Compare symmetric and asymmetric cryptography as used within SSL/TLS and analyse why a secure session needs both.

Step 2 — Symmetric cryptography: Uses a single shared key for both encryption and decryption (e.g., AES-GCM). It is computationally fast and well suited to encrypting large volumes of application data, but requires the two parties to already share the key securely.

Step 3 — Asymmetric cryptography: Uses a mathematically linked public/private key pair (e.g., RSA, ECDHE). It solves the key-distribution problem — the public key can be shared openly — and, via certificates, provides identity authentication, but is computationally expensive for bulk data.

Step 4 — Role during handshake: Asymmetric cryptography authenticates the server (certificate signature) and establishes the shared secret (RSA key wrapping or a Diffie–Hellman exchange), all before any application data flows.

Step 5 — Role during data transfer: Once the shared secret is derived, TLS switches to a symmetric cipher for the bulk of the session, because it is orders of magnitude faster than asymmetric operations at scale.

Step 6 — Comparison:

Property	Symmetric	Asymmetric
Speed	Fast — suited to bulk data	Slow — used sparingly
Key distribution	Hard (needs a secure channel)	Solved via public keys/certificates
Used for	Encrypting session traffic	Authentication and key exchange

Step 7 — Conclusion: Asymmetric cryptography solves the identity and key-distribution problem but is too slow for bulk traffic; symmetric cryptography is fast but needs a securely shared key. TLS uses asymmetric methods to bootstrap trust and a shared secret, then hands off to symmetric encryption for the actual data — each covers the other's weakness.

Q7. Security objectives achieved by TLS

Step 1 — Objective: Analyse how a TLS session delivers confidentiality, integrity, authentication, and secure key exchange.

Step 2 — Confidentiality: Achieved by encrypting application data with the negotiated symmetric session key (e.g., AES-GCM), so only holders of that key can read the traffic.

Step 3 — Integrity: Achieved through the authentication tag of the AEAD cipher (or a separate HMAC in older suites), which detects any modification of the ciphertext in transit.

Step 4 — Authentication: Achieved through the server's CA-signed certificate (and, optionally, a client certificate for mutual TLS), letting each side confirm the other's identity before trusting the channel.

Step 5 — Secure key exchange: Achieved through (ephemeral) Diffie–Hellman or RSA key transport during the handshake, deriving a fresh shared secret without ever transmitting it in the clear; ECDHE additionally provides forward secrecy.

Step 6 — Objective-to-mechanism map:

Objective	TLS mechanism
Confidentiality	Symmetric AEAD encryption of the record layer
Integrity	AEAD authentication tag / HMAC
Authentication	X.509 certificates verified via CA chain
Secure key exchange	(EC)DHE key agreement during the handshake

Step 7 — Conclusion: TLS achieves all four objectives not through one mechanism but through the coordinated handshake-plus-record-layer design, where each phase (authentication, key exchange, then encrypted/integrity-protected transfer) contributes a distinct guarantee.

Q8. Attacker intercepting a TLS handshake — possible attacks and defences

Step 1 — Objective: Analyse what an attacker positioned between browser and server during the TLS handshake could attempt, and how TLS's mechanisms prevent each attempt from succeeding.

Step 2 — Possible attacks: The attacker could try a full man-in-the-middle impersonation, a protocol/cipher downgrade attack (forcing negotiation of a weaker, breakable cipher suite or an older protocol version), certificate spoofing, or replaying captured handshake messages.

Step 3 — Defence against impersonation/spoofing: The attacker would need to present a certificate that verifies against a trusted CA for the target hostname; without the corresponding private key or a CA-signed certificate for that name, signature verification fails and the client aborts the connection.

Step 4 — Defence against downgrade attacks: The Finished message is a MAC computed over the full handshake transcript, sent only after keys are derived; if an attacker altered earlier messages (e.g., stripped strong cipher suites from ClientHello), the transcripts on each side no longer match and the Finished check fails, exposing the tampering.

Step 5 — Defence against replay: Fresh random nonces in ClientHello and ServerHello ensure every handshake derives unique keys, so a captured handshake cannot be replayed to establish a valid session later.

Step 6 — Attack-to-defence map:

Attack	Defence
MITM / certificate spoofing	CA-chain signature verification, hostname check
Cipher/version downgrade	Transcript-hash check in the Finished message
Handshake replay	Per-session random nonces and derived keys

Step 7 — Conclusion: Certificate validation blocks impersonation, transcript integrity checking blocks silent downgrades, and nonce freshness blocks replay — together closing the practical avenues an on-path attacker has during the handshake.

Q9. Architecture and transaction flow of the Secure Electronic Transaction (SET) protocol

Step 1 — Objective: Analyse how SET is structured and how its transaction flow secures an online card payment.

Step 2 — Participants: Cardholder, merchant, payment gateway, the cardholder's issuing bank, the merchant's acquiring bank, and a Certificate Authority hierarchy that issues certificates to all parties.

Step 3 — Architecture: Every participant holds two key pairs — one for digital signatures, one for key exchange/encryption — each backed by a CA-issued certificate, giving SET a fully authenticated, PKI-based structure rather than relying only on a transport-layer channel.

Step 4 — Transaction flow: (1) The cardholder's software initiates a purchase request; (2) it sends the merchant a dual-signed message bundling the Order Information (OI) and Payment Information (PI); (3) the merchant forwards only the PI portion, still encrypted for the gateway, to the payment gateway; (4) the gateway decrypts the PI, authorises the transaction with the issuer over existing banking networks; (5) the merchant receives authorisation and completes/ships the order.

Step 5 — Security guarantee: The merchant never sees the cardholder's raw payment details (only the gateway can decrypt the PI), while the gateway never sees the order contents (only the OI hash), and each party's certificate lets the others verify who they are dealing with.

Step 6 — Simplified transaction flow:

Step	Action
1	Cardholder sends dual-signed OI + PI to merchant
2	Merchant verifies OI, forwards encrypted PI to gateway
3	Gateway decrypts PI, authorises with issuer
4	Gateway returns authorisation to merchant
5	Merchant completes order and confirms to cardholder

Step 7 — Conclusion: SET's PKI-backed, multi-party architecture and its split visibility of order vs payment data give card transactions strong confidentiality and mutual authentication, at the cost of substantially greater infrastructure and coordination than a plain TLS-protected checkout.

Q10. SSL/TLS vs SET for web and payment security

Step 1 — Objective: Compare SSL/TLS and SET on authentication, confidentiality, integrity, payment-specific security, and practical implementation, and identify which suits general web communication.

Step 2 — Comparison table:

Dimension	SSL/TLS	SET
Authentication	Server (optionally client) via one certificate each	All parties (cardholder, merchant, gateway) individually certified
Confidentiality	Whole channel encrypted	Selective — merchant and gateway each see only their portion
Integrity	AEAD/MAC on the channel	Dual signatures binding OI and PI
Payment-specific security	None built in — generic channel security	Purpose-built for card transactions
Practical implementation	Simple, ubiquitous, low overhead	Complex PKI, heavy client software — never widely adopted

Step 3 — Authentication depth: SET's requirement that every participant hold a certificate gives stronger mutual authentication specifically for payment flows, whereas TLS typically authenticates only the server.

Step 4 — Practical adoption: SET's certificate issuance and client-side software requirements proved too costly and complex for mass adoption; TLS, needing only a server certificate, scaled to essentially the entire web.

Step 5 — Conclusion: TLS is the more suitable protocol for general web communication — it is simple, universal, and sufficient for channel security — while SET's more elaborate

payment-specific guarantees were largely superseded in practice by TLS-protected checkout combined with card-network mechanisms such as 3-D Secure.

Q11. SET's use of digital certificates and signatures in an online shopping transaction

Step 1 — Objective: Analyse how SET protects the customer, merchant, and payment gateway in an online purchase using certificates and signatures.

Step 2 — Role of certificates: The cardholder, merchant, and gateway each hold a certificate chained to the SET CA hierarchy, letting every party cryptographically verify who the others are before exchanging sensitive data.

Step 3 — Protecting the customer: The customer's payment information is encrypted so that only the gateway's key can decrypt it; the merchant, who is not a trusted custodian of card data, never sees it, limiting exposure if the merchant is compromised.

Step 4 — Protecting the merchant: The customer's dual-signed order confirms both the order details and the customer's authorisation of payment, giving the merchant verifiable proof of a legitimate order without needing to handle or store card data itself, reducing its fraud and compliance liability.

Step 5 — Protecting the payment gateway: The gateway verifies the signatures from both customer and merchant before authorising the transaction with the issuer, ensuring it only acts on requests that are provably linked to a matching, unaltered order.

Step 6 — Conclusion: By combining per-party certificates with dual-signed messages, SET ensures each participant gets exactly the information it needs and no more, while every party can verify authenticity — protecting the customer's card data, the merchant's liability, and the gateway's authorisation integrity simultaneously.

Q12. Dual signatures in SET

Step 1 — Objective: Explain the dual-signature construction and analyse how it gives confidentiality between customer, merchant, and gateway while preserving transaction integrity.

Step 2 — Construction: The customer computes hash(OI) and hash(PI) separately, concatenates the two hashes, hashes the result again, and signs that final hash with their private key — producing the Dual Signature (DS).

Step 3 — What the merchant receives: The merchant receives the OI in the clear, plus hash(PI) and the DS, but never the PI itself; it verifies the DS by recomputing hash(OI), combining it with the supplied hash(PI), and checking against the signature.

Step 4 — What the gateway receives: The gateway receives the PI, plus hash(OI) and the DS, but never the OI itself; it verifies the DS the same way, using the supplied hash(OI) combined with its own computed hash(PI).

Step 5 — Confidentiality achieved: Because each party only ever holds a hash (not the plaintext) of the portion meant for the other party, neither the merchant nor the gateway can recover information they are not authorised to see, even though both are validating the same signature.

Step 6 — Integrity achieved: Since the DS is computed over the combined hash of both OI and PI, any tampering with either piece — by any party or an attacker — breaks the signature check for both the merchant and the gateway, cryptographically linking the order and the payment as belonging to the same, unaltered transaction.

Step 7 — Conclusion: The dual signature lets two independently held pieces of a single transaction be verified by two different parties without either seeing the other's data, achieving selective confidentiality alongside a single, tamper-evident binding between order and payment.

Q13. Viruses, worms, and Trojans — a comparative analysis

Step 1 — Objective: Differentiate viruses, worms, and Trojans by propagation mechanism, host dependency, detection difficulty, and potential damage.

Step 2 — Comparison table:

Property	**Virus**	**Worm**	**Trojan**
Propagation	Attaches to a host file/program; spreads when the infected file is shared or run	Self-replicates across a network by exploiting vulnerabilities or services	Does not self-replicate; relies on the user installing it, disguised as legitimate software
Host dependency	Requires a host program to attach to	Independent — does not need a host file	Independent — is itself the (disguised) program
Detection challenge	Signature-detectable once known; polymorphic variants evade static signatures	Rapid spread makes containment time-critical; scanning behaviour can trigger network-based detection	Blends in as legitimate software; often evades detection until its payload activates
Potential damage	Corrupts/deletes files, degrades host performance	Consumes bandwidth/resources at scale, can carry a damaging payload across many hosts quickly	Opens backdoors, exfiltrates data, or grants remote control, often silently

Step 3 — Conclusion: The three differ chiefly in how they get onto and move between systems — viruses need a host and human action to spread, worms spread autonomously across networks, and Trojans rely on deception rather than replication — which is why each demands a different mix of detection and containment strategy.

Q14. University network incident — virus, worm, or Trojan?

Step 1 — Objective: Diagnose whether sudden network-wide traffic spikes and multiple simultaneous slowdowns are more consistent with a virus, a worm, or a Trojan.

Step 2 — Symptom analysis: The defining symptoms are rapid, network-wide traffic growth and slowdowns across multiple systems simultaneously, not just corruption on one machine.

Step 3 — Why not a virus: A virus needs an infected file to be individually shared or executed on each new host, which produces a comparatively slow, file-by-file spread rather than a sudden network-wide traffic surge.

Step 4 — Why not (typically) a Trojan: A Trojan usually operates covertly on individual infected hosts and does not self-propagate; it would not, by itself, explain rapid spread and slowdowns across many systems unless it were part of a coordinated botnet — a less direct explanation than a worm.

Step 5 — Why a worm fits: A worm self-replicates across the network without needing a user to run anything, scanning for and exploiting vulnerable hosts; this autonomous, exponential spread naturally produces exactly the symptoms observed — sudden bandwidth consumption and simultaneous slowdowns across many machines.

Step 6 — Conclusion: The incident is most consistent with a worm outbreak; recommended response includes isolating affected network segments, patching the exploited vulnerability, and deploying IDS/IPS to detect and block further propagation.

Q15. How a worm spreads without user interaction, and containment measures

Step 1 — Objective: Analyse the mechanism by which a worm propagates autonomously and identify measures to prevent and contain such attacks.

Step 2 — Exploitation mechanism: A worm targets a vulnerability in a network-facing service (e.g., a buffer overflow in an open port), allowing it to execute code on a remote host without any file being opened or link being clicked by a user.

Step 3 — Propagation model: Once running on a host, the worm scans a range of IP addresses/ports for other vulnerable machines, exploits them the same way, and copies itself over — a self-sustaining, exponential spread with no human step in the loop.

Step 4 — Prevention measures: Timely patch management to close the exploited vulnerability, disabling unused/unnecessary network services, and network segmentation to limit how far a single compromised host can scan and reach.

Step 5 — Containment measures: IDS/IPS to detect and block the exploit traffic and scanning pattern in real time, rate-limiting outbound connections from hosts, and honeypots/decoy hosts to detect and study propagation attempts early.

Step 6 — Conclusion: Because worm propagation requires no user action, defence must be structural — patching, segmentation, and inline detection/blocking — rather than relying on user awareness, which is the primary defence against viruses and Trojans.

Q16. Trojan-based unauthorized access — attack analysis

Step 1 — Objective: Analyse a scenario where a user downloads an apparently legitimate application that secretly grants unauthorized access, and identify what distinguishes a Trojan from a virus or worm.

Step 2 — Attack analysis: The application performs its advertised function (or appears to) while covertly installing a backdoor or remote-access component, giving the attacker unauthorized entry once the user runs it — a classic Trojan payload delivered through social engineering rather than exploitation of a network flaw.

Step 3 — Distinguishing from a virus: A virus needs to attach itself to another host program and spreads when that host file is shared; the Trojan here is itself the whole (disguised) program and does not attach to or infect other files.

Step 4 — Distinguishing from a worm: A worm self-replicates and spreads autonomously across a network without user action; this Trojan does neither — it relies entirely on the user being deceived into installing it once, and does not copy itself onward.

Step 5 — Characteristic table:

Characteristic	Trojan
Self-replication	No
Requires user deception	Yes — disguised as legitimate software
Typical payload	Backdoor / remote access / data theft
Spread mechanism	Manual download/install by the victim, not autonomous propagation

Step 6 — Conclusion: The absence of self-replication combined with reliance on user deception to gain initial execution is what marks this as a Trojan rather than a virus or worm.

Q17. Role of firewalls in protecting an internal network

Step 1 — Objective: Analyse how firewalls protect an organisation's internal network from external threats.

Step 2 — Packet filtering: Examines each packet's header fields (source/destination IP, port, protocol) against a rule set and permits or drops it accordingly, blocking traffic from unauthorized sources or to disallowed services before it reaches internal hosts.

Step 3 — Access control: Enforces organisational policy about which internal services may be reached from outside (and which internal hosts may reach the outside), implementing a least-privilege boundary between trusted and untrusted networks.

Step 4 — Traffic monitoring: Logs allowed and denied connection attempts, providing visibility into scanning attempts, policy violations, and a forensic trail for incident investigation.

Step 5 — Combined contribution: Together, filtering blocks unwanted traffic at the boundary, access control ensures only intended services are exposed, and monitoring provides the ongoing awareness needed to detect and respond to attack attempts — a firewall is as much a policy-enforcement and visibility tool as a blocking one.

Step 6 — Conclusion: A firewall's value comes from combining rule-based blocking with policy enforcement and logging, giving an organisation a controlled, observable boundary between its internal network and the external world.

Q18. Comparing firewall types

Step 1 — Objective: Compare packet-filtering, stateful inspection, proxy/application-level, and next-generation firewalls, analysing the advantages and limitations of each.

Step 2 — Comparison table:

Type	Advantages	Limitations
Packet-filtering	Fast, low overhead, simple to deploy	No awareness of connection state or payload content; vulnerable to spoofing and crafted packets
Stateful inspection	Tracks connection state, blocks packets that don't belong to a legitimate session	Still limited visibility into application-layer payload
Proxy / application-level	Inspects and mediates traffic at the application layer, can enforce content-specific policy	Higher latency/overhead; needs a proxy per protocol/application
Next-generation (NGFW)	Combines stateful inspection with deep packet inspection, intrusion prevention, and application awareness	Most complex and resource-intensive; requires careful

Type	Advantages	Limitations
		tuning to avoid false positives/performance loss

Step 3 — Analysis: Each generation adds visibility at the cost of complexity and processing overhead: packet filters see only headers, stateful firewalls add connection context, proxies add application-layer mediation, and NGFWs integrate all of this plus payload-level threat detection.

Step 4 — Conclusion: The right choice is a trade-off between required depth of inspection and acceptable overhead — modern deployments generally favour NGFWs at the perimeter, often still combined with simpler stateless filters for coarse, high-speed blocking.

Q19. Firewall architecture with DMZ for web, mail, database, and internal servers

Step 1 — Objective: Design a firewall architecture that protects a web server, mail server, database server, and internal employee network, and justify the placement of the DMZ and firewall components.

Step 2 — Requirement analysis: The web and mail servers must be reachable from the Internet and so carry the highest exposure; the database server must never be directly reachable from the Internet; the internal employee network must be shielded from both the Internet and the public-facing servers.

Step 3 — Architecture — screened-subnet (three-legged) design: Use two firewall layers forming three zones: an external firewall between the Internet and a DMZ, the DMZ itself hosting the web and mail servers, and an internal firewall between the DMZ and the internal network (which contains the database server and employee workstations).

Step 4 — Placement justification: The external firewall permits only the specific ports the web/mail servers need (e.g., 443, 25/587) from the Internet into the DMZ. The internal firewall permits connections only from the DMZ's application servers to the database server on its specific port, and blocks any direct Internet-to-database or Internet-to-internal-network path entirely — so a compromise of a DMZ server does not automatically expose the database or employee network.

Step 5 — Additional controls: Deploy IDS/IPS at both firewall boundaries, apply least-privilege rule sets (default-deny with explicit allows), and keep the database server on a network segment with no route to the Internet.

Step 6 — Zone summary:

Zone	Contains	Reachable from
External	Internet	—
DMZ	Web server, mail server	Internet (specific ports) and internal firewall only

Zone	Contains	Reachable from
Internal network	Database server, employee workstations	DMZ application tier only (never directly from the Internet)

Step 7 — Conclusion: A two-firewall, screened-subnet architecture isolates public-facing services in a DMZ while keeping the database and internal network behind a second control point, so a single compromised perimeter service does not directly expose the organisation's most sensitive assets.

Q20. Stateful vs stateless firewalls for TCP connections

Step 1 — Objective: Analyse how a stateful firewall differs from a stateless packet-filtering firewall in handling TCP connections, and explain why stateful inspection is stronger.

Step 2 — Stateless filtering: Evaluates each packet independently against static rules (source/destination IP and port), with no memory of prior packets — it cannot tell whether a packet is part of an established, legitimate connection or an unsolicited one crafted to look similar.

Step 3 — Stateful inspection: Maintains a connection-state table tracking the TCP handshake (SYN, SYN-ACK, ESTABLISHED) for each session, and only permits packets that correspond to a recognised, in-progress connection.

Step 4 — Why stateful is stronger: A stateless firewall can be tricked by packets with plausible header values (e.g., an ACK-flagged packet with no matching prior SYN) that a stateful firewall would immediately reject as out-of-state, closing off techniques like certain port scans and session-hijacking attempts that rely on injecting packets into or around a connection.

Step 5 — Conclusion: By understanding TCP's connection lifecycle rather than judging packets in isolation, stateful inspection distinguishes legitimate session traffic from crafted or out-of-context packets, providing materially stronger protection than stateless filtering at a modest cost in state-tracking overhead.

Q21. Why a firewall alone is insufficient — the role of IDS/IPS

Step 1 — Objective: Analyse why a firewall that correctly blocks unauthorized traffic can still be bypassed by an attacker using permitted traffic, and explain how IDS/IPS complements it.

Step 2 — Why the firewall alone is insufficient: A firewall's decisions are based on connection-level attributes (address, port, protocol state); it does not typically inspect the content of allowed traffic, so an attack embedded inside otherwise-permitted traffic (e.g., a malicious payload over HTTP, or SQL injection in a web request) passes through untouched.

Step 3 — Role of an IDS: Monitors traffic (or host activity) for known attack signatures or anomalous behaviour within permitted flows, generating alerts when it recognises exploitation attempts that the firewall had no reason to block.

Step 4 — Role of an IPS: Performs the same detection as an IDS but sits inline in the traffic path, allowing it to actively drop or block malicious traffic in real time rather than only alerting after the fact.

Step 5 — Layered defence: Placing IDS/IPS behind (or alongside) the firewall adds a content- and behaviour-aware inspection layer that catches exactly the class of attack that address/port-based filtering cannot — the firewall narrows what traffic is allowed through at all, and IDS/IPS scrutinises what that allowed traffic is actually doing.

Step 6 — Conclusion: Because a firewall and an IDS/IPS operate at different levels of inspection (connection attributes vs payload/behaviour), combining them closes the gap that either control leaves open on its own.

Q22. IDS vs IPS

Step 1 — Objective: Compare Intrusion Detection Systems and Intrusion Prevention Systems on deployment, detection, response, traffic handling, and security impact.

Step 2 — Comparison table:

Dimension	**IDS**	**IPS**
Deployment	Out-of-band (monitors a copy of traffic)	Inline (sits directly in the traffic path)
Detection	Signature- and/or anomaly-based analysis	Same detection techniques as IDS
Response	Generates alerts for human/SOC review	Can actively drop, block, or reset malicious traffic automatically
Traffic handling	Does not affect the live traffic flow	Directly affects traffic flow; a false positive can block legitimate traffic
Security impact	Improves visibility and forensic capability	Improves real-time containment, but introduces a single point that must be highly accurate and available

Step 3 — Conclusion: IDS trades immediacy for safety (it never itself disrupts traffic), while IPS trades a small risk of blocking legitimate traffic for the ability to stop an attack in progress — many deployments run both, using IDS insights to tune IPS rules before enabling active blocking.

Q23. Signature-based vs anomaly-based intrusion detection

Step 1 — Objective: Analyse signature-based and anomaly-based IDS techniques and their effectiveness against known and zero-day attacks, and their false-positive/false-negative behaviour.

Step 2 — Signature-based detection: Matches traffic or activity against a database of known attack patterns; highly accurate and low false-positive against attacks already catalogued, but structurally blind to zero-day attacks with no matching signature, producing false negatives for novel threats.

Step 3 — Anomaly-based detection: Builds a baseline of normal behaviour and flags significant deviations from it; capable of catching previously unseen (zero-day) attacks because it does not depend on a known pattern, but tends to produce more false positives when legitimate behaviour shifts, and requires careful baseline tuning.

Step 4 — Effectiveness summary:

Technique	Known attacks	Zero-day attacks	False positives	False negatives
Signature-based	High detection accuracy	Cannot detect (no signature)	Low	High for novel attacks
Anomaly-based	Can detect if it deviates from baseline	Can detect via behavioural deviation	Higher — legitimate changes can trigger alerts	Lower for genuinely novel attacks

Step 5 — Conclusion: The two techniques have complementary blind spots — signature-based detection is precise but only for what is already known, anomaly-based detection generalises to the unknown at the cost of noisier alerts — so a hybrid approach using both typically gives the strongest overall coverage.

Q24. Repeated failed logins followed by unusual traffic — IDS detection and IPS response

Step 1 — Objective: Analyse how an IDS could detect this activity and how an IPS could respond, for a company observing repeated unsuccessful login attempts followed by unusual outbound network traffic from an internal computer.

Step 2 — IDS detection of the login attempts: A signature/rule matching a threshold of failed authentication attempts within a short window (a classic brute-force pattern) would trigger an IDS alert on that host.

Step 3 — IDS detection of the follow-on traffic: An anomaly-based component comparing the host's outbound traffic volume/destination against its normal baseline would flag the subsequent unusual traffic as a deviation, consistent with a successful compromise establishing command-and-control communication or exfiltration.

Step 4 — IPS response: Positioned inline, the IPS could automatically throttle or block further authentication attempts from the offending source once the brute-force threshold is crossed, and, on detecting the anomalous outbound traffic, terminate or quarantine that internal host's network access to contain further spread or data loss.

Step 5 — Conclusion: The two-stage pattern — signature-based detection of the brute-force attempt followed by anomaly-based detection of the resulting anomalous traffic — illustrates why combining both detection techniques with an inline (IPS) response gives both early warning and active containment of a developing compromise.

Q25. Comprehensive web security architecture for an online banking system

Step 1 — Objective: Design a layered security architecture for an online banking system using TLS, digital certificates, firewalls, IDS/IPS, and malware protection, and analyse each mechanism's contribution.

Step 2 — Perimeter and network segmentation: A screened-subnet firewall architecture places the public-facing web/application tier in a DMZ, with a second firewall isolating the database and core banking systems from both the Internet and the DMZ, following the same zoning principle as Q19.

Step 3 — Transport security: TLS, backed by a CA-issued digital certificate for the bank's domain, encrypts and integrity-protects every customer session, authenticating the server to the customer and protecting credentials and transaction data in transit, as analysed in Q4 and Q7.

Step 4 — Intrusion detection/prevention: IDS/IPS deployed at the perimeter and between network segments inspects allowed traffic for exploitation attempts and anomalous behaviour (Q21–Q24), catching attacks that firewall rules alone would permit through.

Step 5 — Malware protection: Endpoint protection (anti-malware/EDR) on servers and employee machines, combined with email/attachment scanning and disciplined patch management, defends against virus, worm, and Trojan infection vectors (Q13–Q16) that could otherwise bypass network-layer controls entirely.

Step 6 — Application-layer and payment security: Server-side input validation and a web application firewall reduce the payload-level attacks that plain TLS does not address, while transaction authentication follows the same principle SET uses (Q9–Q12) — sensitive data is exposed only to the parties that need it, each cryptographically verified.

Step 7 — Layer-to-threat mapping:

Layer	Mechanism	Threat mitigated
Network perimeter	Segmented firewalls / DMZ	Unauthorized network access to core systems

Layer	Mechanism	Threat mitigated
Transport	TLS + digital certificates	Eavesdropping, tampering, server impersonation
Traffic inspection	IDS/IPS	Exploits and anomalies hidden in permitted traffic
Endpoint	Anti-malware / patching	Virus, worm, Trojan infection
Application/payment	Input validation, WAF, verified transaction data	Injection attacks, payment fraud

Step 8 — Conclusion: No single control secures an online banking system end to end; the architecture's strength comes from each layer covering a gap the others leave open — network segmentation limits reachability, TLS secures the channel, IDS/IPS watches what the firewall must allow through, and endpoint/application controls address threats that never touch the network boundary at all.